

COMP521 Assignment 2 — Linux Thread Synchronization and Race-Condition Analysis Lab

Purpose

The main purpose of this program is to demonstrate how multiple threads accessing the same data can lead to a race condition and how synchronization can prevent incorrect results. The program can be run in two ways. Synchronized and unsynchronized. The unsynchronized version does not implement any protection measures that would protect the data from being updated by multiple threads at the same time. The synchronized version implements POSIX mutex locks to ensure that only one thread can update the data at a time.

Threads within the same process can access shared memory, which makes communication between threads efficient but also creates synchronization problems when multiple threads modify the same data. In this case, the variable being updated is **counter**. Each of the four threads performed 1,000,000 increments, so the expected final value was 4,000,000.

Evidence Table - Unsynchronized

Trial	Expected	Actual	Correct?
1	4,000,000	1,015,666	No
2	4,000,000	1,038,877	No
3	4,000,000	1,075,080	No
4	4,000,000	1,054,919	No
5	4,000,000	1,062,836	No
6	4,000,000	1,132,549	No
7	4,000,000	1,033,014	No
8	4,000,000	1,089,762	No
9	4,000,000	1,125,835	No
10	4,000,000	1,066,355	No

Evidence Table - Synchronized

Trial	Expected	Actual	Correct?
1	4,000,000	4,000,000	Yes
2	4,000,000	4,000,000	Yes
3	4,000,000	4,000,000	Yes
4	4,000,000	4,000,000	Yes
5	4,000,000	4,000,000	Yes

Trial	Expected	Actual	Correct?
6	4,000,000	4,000,000	Yes
7	4,000,000	4,000,000	Yes
8	4,000,000	4,000,000	Yes
9	4,000,000	4,000,000	Yes
10	4,000,000	4,000,000	Yes

Unsynchronized Experiment

The program created four threads. Each thread executed the increment operation 1,000,000 times. Because there were four threads, the expected final value of the shared counter should be 4,000,000. When synchronization was disabled, the threads were allowed to update the shared counter without a mutex. The results showed a significant difference between the expected and actual values. For example, Trial 1 produced 1,015,666 instead of 4,000,000, while Trial 6 produced 1,132,549.

This is clearly not the expected result and there is clear evidence to show that a lot of updates were lost. The exact value was not identical between trials, which demonstrates that the result depended on the particular execution and scheduling of the threads.

Race-Condition Analysis

For this experience, we used a shared variable `counter`. All of the four threads accessed and modified the `counter` variable. The POSIX mutex allowed us to create a critical section for the shared counter, which ensured that only one thread at a time could update the shared counter. Without the mutex, there is no guarantee that the counter will be updated correctly.

For example, suppose `counter` contains the value 100. Thread A could read 100 while Thread B also reads 100 before either thread writes its result. Both threads would then calculate 101. If Thread A writes 101 and Thread B subsequently writes 101, the counter has increased by only one even though two increments were performed. Thread B's update effectively overwrote Thread A's update. This is called a lost update.

This race condition occurs because the threads are accessing shared data without ensuring that the read-modify-write sequence is performed safely. The operating system scheduler can switch between threads at different points during execution, so the exact order of operations is not guaranteed.

Mutex-Protected Experiment

For the second experiment, synchronization was enabled by setting `use_mutex` to 1. We use the POSIX mutex to protect the counter update.

```
pthread_mutex_lock(&lock);  
counter++;  
pthread_mutex_unlock(&lock);
```

`pthread_mutex_lock()` requires a thread to acquire the mutex before entering the protected section. If another thread already owns the mutex, that thread must wait. After the counter has been updated, `pthread_mutex_unlock()` releases the mutex so another thread can enter the critical section.

This provides mutual exclusion. Although all four threads can still execute concurrently, only one thread at a time can perform the protected counter update. Therefore, two threads cannot simultaneously read the same old counter value and overwrite each other's updates.

Evidence Comparison

The difference between the two experiments was the synchronization of the shared counter update. In the unsynchronized experiment, no mutex protected `counter++`. In the synchronized experiment, the mutex was locked before the critical section and unlocked afterward. The results were substantially different. The unsynchronized trials produced values between 1,015,666 and 1,132,549, and none of the ten trials reached the expected 4,000,000. Every synchronized trial produced exactly 4,000,000.

This evidence shows that the mutex did more than simply change the final output. It changed the way threads were allowed to access the shared resource. Without synchronization, the threads could interleave their read, modify, and write operations in a way that caused lost updates. With the mutex, access to the critical section was mutually exclusive, preventing conflicting updates.

Why Repeated Trials Matter

Running the unsynchronized program only once would not be sufficient evidence that the program was correct. Thread scheduling is nondeterministic, meaning that the exact order in which threads execute their instructions can change from one execution to another. A particular execution might happen to produce the expected result even when a race condition exists. Another execution could produce an incorrect result because a different thread interleaving exposes the race. Therefore, a single successful run cannot prove that the program is correctly synchronized.

Conclusion

This experiment demonstrated the challenges of using shared memory with multiple threads. Four threads were able to access and modify the same counter variable because they belonged to the same process. However, allowing all four threads to execute `counter++` without synchronization resulted in a race condition and lost updates.